

Laboratorio II

Corso A

Lezione 5

- Programmazione C di base
 - Array multidimensionali
 - Parametri dalla linea di comando
- Prima esercitazione

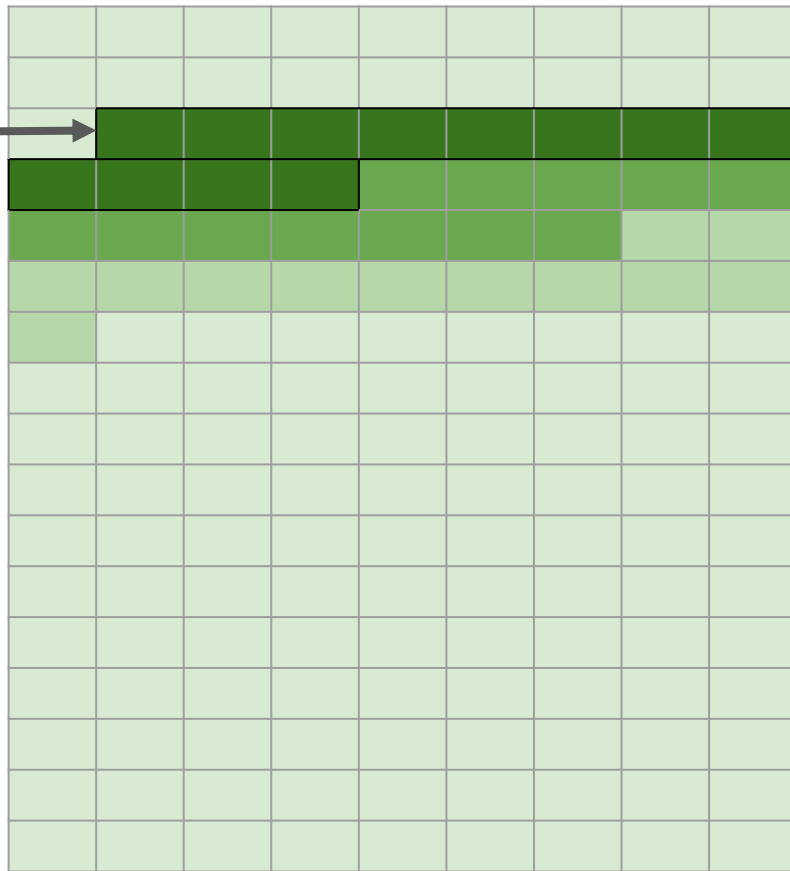
Array bidimensionali

```
int a[3][3];
```

- crea una variabile `a`
- alloca in memoria sullo stack 9 interi contigui
- memorizza in `a` l'indirizzo dell'inizio della zona di memoria allocate
- viene allocata per righe
- usato per le **matrici**
- La seconda dimensione è molto importante (aiuta il compilatore a calcolare la locazione di memoria giusta, dati gli indici)
- Si possono definire array con quante dimensioni vogliamo

```
int a[3][4][2];
```

a



Array bidimensionali

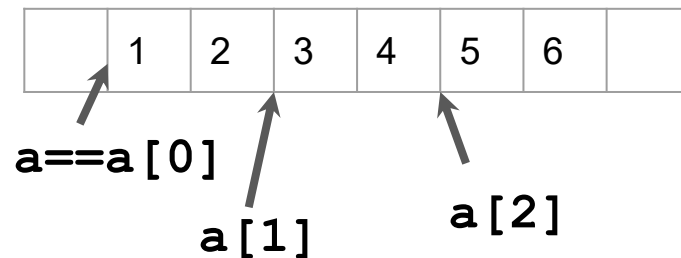
```
#include <stdio.h>

int main(void) {

    int a[3][2]={1,2},{3,4},{5,6}};

    printf("%p\n", a);
    printf("%p, %p, %p\n", a[0], a[1], a[2]);

}
```



0x7ffd2ba5ac10

0x7ffd2ba5ac10, 0x7ffd2ba5ac18, 0x7ffd2ba5ac20

Tipi di dato custom - enum

- `enum` definisce un tipo di dato enumerativo, usato per rendere autodocumentante il codice
- Esempio:
 - `enum giorni {lunedì, martedì, mercoledì, giovedì, venerdì, sabato, domenica};`
- Ogni identificatore è una costante intera che, per default, parte dal valore 0 e incrementa di 1. Si può usare ovunque sia possibile usare una costante di tale tipo. Esempio:

```
enum giorni giorno;  
switch(giorno):  
{  
    case lunedì:  
        printf("lunedì");  
        break;  
    case martedì:  
        printf("martedì");  
        break;  
    ...  
    default:  
        printf("domenica");  
}
```

- Si può cambiare i valori assegnati agli identificatori:
 - `enum giorni {lunedì = 1, martedì, mercoledì, giovedì=7, venerdì, sabato, domenica};`

Tipi di dato custom - typedef

- `typedef` si può usare per rinominare dei tipi esistenti, ad esempio nel caso di nomi di tipo lunghi, o per aggiungere nomi maggiormente mnemonici.
- Si associa ad un identificatore, come nei seguenti casi, e si usa l'identificatore come sostitutivo lessicale al tipo indicato:

```
typedef char[] String;
```

- Le keyword `struct` e `union` vanno sempre ripetute quando si dichiara una nuova variabile.
- E' comodo dare un nome più corto al nuovo tipo di dato:

```
typedef struct {...} Studente;
```

```
typedef enum {...} Giorni;
```

Parametri dalla linea di comando

- Possiamo passare al programma dei parametri e opzioni indicate durante la sua invocazione a linea di comando
 - Simile alla chiamata della shell di comandi UNIX
- Esempio: nel calcolo del max passiamo i due valori di *a* e *b* dalla linea di comando

```
./main 7 5    - stampa 7
```

```
./main 2 3    - stampa 3
```

Parametri dalla linea di comando

Vengono passati al programma tramite parametri della funzione `main`:

```
int main(int argc, char argv[][])
```

`argc` - numero di parametri

`argv` - parametri stessi – stringhe

`argv[0]` - nome del programma

Domande?



Esercizi

- Scrivere un programma che legge due vettori di dimensione ***n*** fissa, ad esempio 10, e fa il prodotto scalare tra i due vettori. Il prodotto scalare è la somma dei prodotti di ogni coppia di elementi dei due vettori dello stesso indice, cioè $a[0]*b[0] + \dots + a[n]*b[n]$
- Scrivere un programma che legge ***n*** e due matrici quadrate di dimensione ***nXn***, ad esempio di 10 elementi, e fa il prodotto scalare tra le due matrici.
- Scrivere un programma che fa la somma dei parametri ricevuti dalla linea di comando

Esercizi

- Scrivere un programma che inverta gli elementi di un array di interi A, di dimensione 10. Il programma deve invertire l'array senza utilizzare un array di appoggio, ossia scambiare il contenuto della prima e dell'ultima cella, della seconda e della penultima, ecc... ed infine stampare a video ogni singolo elemento dell'array modificato.
- Si scriva un programma di gestione dell'anagrafe dei dipendenti di un'azienda che legga i seguenti dati per 6 diversi dipendenti:
 - Un codice identificativo del dipendente (intero), **diverso** per ogni dipendente
 - L'età del dipendente in anni (un intero)
 - Stipendio (un float, indicato in euro come segue: 16540.32);
 - Il tipo di dipendente, indicando con il numero 0 i dirigenti, 1 i quadri, 2 gli operai e 3 gli impiegati, usando una opportuna `enum` con etichette rispettivamente **dirigente**, **quadro**, **operaio** e **impiegato**.

Il numero di dipendenti da inserire è N fisso, ad esempio 20; per ogni dipendente, si chieda in input il codice identificativo, controllando se è diverso da uno già immesso e richiedendolo se è diverso, l'età, lo stipendio e il tipo di dipendente.

Nello stesso ordine in cui sono stati immessi i dati dei dipendenti si stampino, uno per linea, il codice identificativo, l'età e il ruolo di ogni dipendente (l'etichetta) che guadagni meno dello stipendio medio aritmetico di tutti i dipendenti dell'azienda. I valori stampati devono essere separati da un unico spazio.

Esempio di Input:

35,1,56,55650.33, 0, 2, 28, 36540.56, 1, 3, 22, 22120.54, 2, 4, 32, 28450.45, 3, 5, 45, 18960.65, 2, 6, 33,38960.54, 1

Esempio di Output:

3 22 operaio

4 32 impiegato

Esercizi: soluzione 1

```
#include <stdio.h>
int main()
{
    int i,j,scambio;
    int A[10];

    for ( i = 0; i < 10; i++ )
        A[i]=i;

    /* Inversione, scorro l'array fino alla metà della sua dimensione */
    for ( i = 0; i < 10/2; i++ )
    {
        j = (10-1)-i;
        scambio = A[i];
        A[i] = A[j];
        A[j] = scambio;
    }

    /* Output */
    for ( i = 0; i < 10; i++ )
        printf("%d\n", A[i]);
    return 0;
}
```

Esercizi: soluzione 2

```
#include <stdio.h>
#include <stdlib.h>

typedef enum
{
    dirigente,
    quadro,
    operaio,
    impiegato
} ruolo_t;

typedef struct
{
    int id;
    int eta;
    float salario;
    ruolo_t ruolo;
} dipendente_t;

int main(int argc, char *argv[])
{
    int i, id_dip, eta_dip, ruolo_dip, eta;
    float media = 0, salario_dip;

    dipendente_t dipendenti[6];
    scanf("%d",&eta);
```

```
    for (i=0; i<6; i++)
    {
        scanf("%d",&id_dip);
        scanf("%d",&eta_dip);
        scanf("%f",&salario_dip);
        scanf("%d",&ruolo_dip);
        dipendenti[i].id = id_dip;
        dipendenti[i].eta = eta_dip;
        dipendenti[i].salario = salario_dip;
        dipendenti[i].ruolo = ruolo_dip;

        /* calcolo della media, fatto durante l'inserimento */
        media += salario_dip;
    }

    media /= 6;

    for (i=0; i<6; i++)
        if (dipendenti[i].salario < media && dipendenti[i].eta <= eta){
            printf("%d ", dipendenti[i].id);
            printf("%d ", dipendenti[i].eta);
            switch(dipendenti[i].ruolo){
                case dirigente:
                    printf("dirigente\n");
                    break;
                case quadro:
                    printf("quadro\n");
                    break;
                case operaio:
                    printf("operaio\n");
                    break;
                default:
                    printf("impiegato\n");
            }
        }

    return 0;
}
```